o

UART Communications

Group #6: Hanna Estrada , Justin Narciso, Natasha Kho
Computer Engineering / CSULB
Embedded Systems III
Min He
03/16/2026

1. Introduction....3
2. System Design and Architecture....5
3. Development Steps and Task Assignments....6
4. Hardware Design....8
Schematic....8
Bill of Materials....8
5. Software Design....9
6. Demonstration....21
8. Conclusion....23
9. References....25

Table of Contents

1. Introduction

The UART Communication Project uses PWM, GPIO buttons and LED, and three different UART modules. Interrupts are used for the GPIO buttons and the UART modules. In this project, we used UART modules UART0, UART5, and UART7. The goal of this project is is to create a communication system between two MCUs via UART communication. In order to achieve this goal, we will implement three different modes of communication between PC and MCU, MCU and MCU, and MCU to another PC.

In this project, we set up a main menu to handle entering and exiting each mode. This is done via FSM, which controls current mode and ensures that two modes cannot be active at the same time. More FSMs are used each mode: Mode 1 has its own submenu to control which menu option is chosen, Mode 2 and Mode 3 each have one to determine waiting or sending state between MCU1 and MCU2.

Mode 1 focuses on the communication between PC and MCU. In this Mode, a submenu is brought up that gives the user several options: change LED color, change LED brightness, and exit mode. The user must type in the corresponding number on their PC terminal, and upon clicking enter, will cause the MCU to present the option to the user. The first option, change LED color, allows the user to change the LED color of the MCU if the user enters a valid char. This change is also reflected on the hardware, as the user can see the corresponding LED color light up on the MCU. The second option, change LED brightness, allows the user to enter a number from 0 to 100, which changes the LED brightness to the corresponding percentage. The change can be seen on the hardware by the LED getting brighter or dimmer depending on the user's input. The final option, exit mode, exits Mode 1 and sends the user back to the Main Menu.

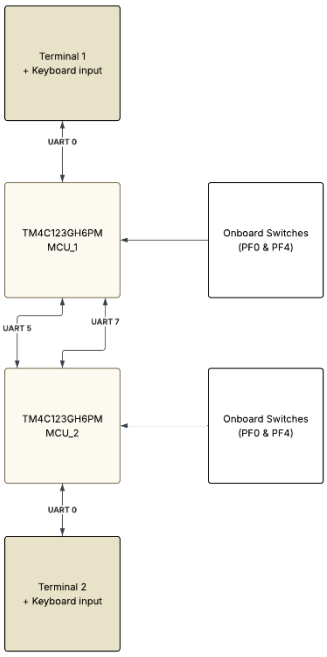
Mode 2 focuses on the communication between MCU to MCU. In this Mode, the MCU1 begins as the active MCU. On the active MCU, the user can press SW2 to cycle through all the possible colors in a round robin fashion. The current color is shown on the corresponding PC when the color is changed. On the active MCU, the user can press SW1 to send the color to the waiting MCU. This will change the current color of the waiting MCU. After pressing SW1, the waiting MCU and sending MCU will change states, inverting which MCU is sending and receiving data. The two MCUs can send data back and forth until the mode is exited. This occurs when, exclusively on PC1, the user enters the '^' and clicks enter. This will interrupt the system and cause both MCUs to exit Mode 2.

Mode 3 focuses on the communication between PC and PC, using the MCUs as a communication device. In this mode, the user is prompted to send messages to each other. PC1 begins as the transmitter, able to send messages to PC2. The message must not contain more than 30 characters, which is handled by cutting off any characters about the 30 limit. Once the user at PC1 clicks enter, the message is transmitted from PC1 to MCU1 to MCU2 to PC2, and the message appears on PC2's terminal. After this, PC1 is set to receiving mode and PC2 can now transmit messages to PC1. This goes back and forth until the mode is exited by pressed SW1 on either MCU.

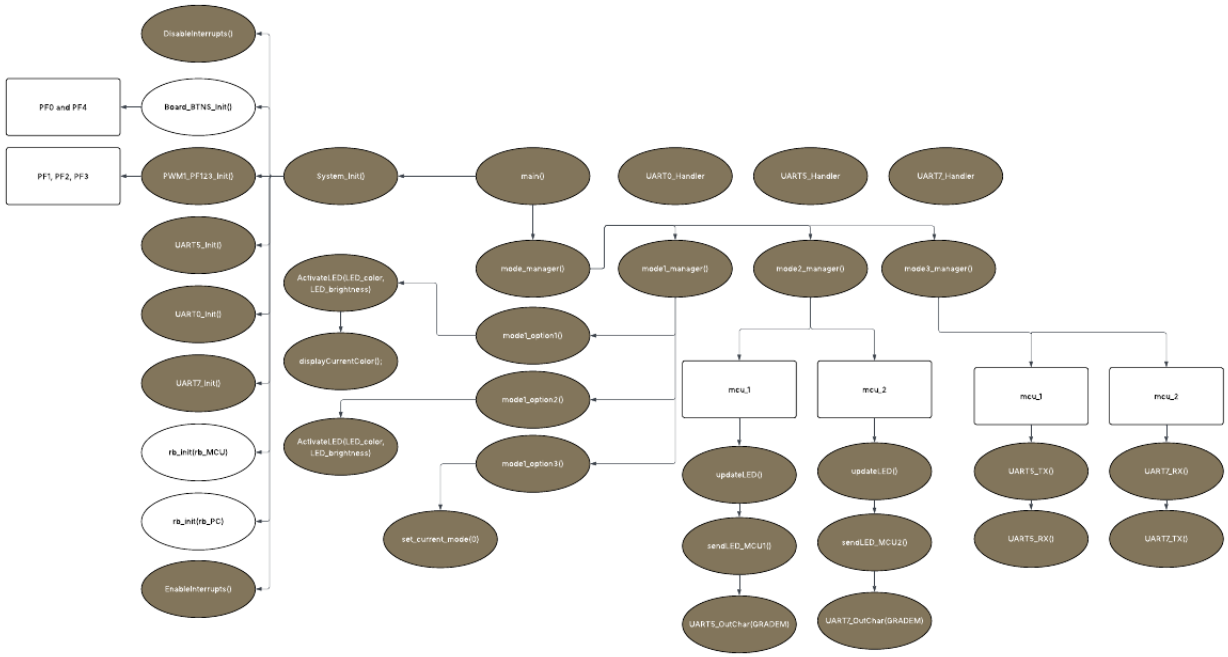
Ultimately, this system is made of two systems that communicate with each other and are interconnected. It provided a good challenge to ensure both MCUs are able to access the same

modes at the same time. While the MCU to MCU communication as a function is only part of Mode 2 and Mode 3, it is important to the entire system, as the two MCUs are often sending each other flags in order to ensure they exit certain modes at the same time. There is also the matter of PC to MCU communication. PC users may enter invalid inputs, which must be addressed to ensure that the system does not break. The two systems we made work in tandem to communicate with each other.

2. System Design and Architecture



This system uses two TM4CGH6PM microcontrollers and one to two laptops. The two microcontrollers communicate with each other using UART 5 and UART 7 respectively. They use the same baud rate – 80,000. To communicate with their respective terminals, they use different baud rates. UART 5 to UART 0 uses 921,600 and UART 7 to UART 0 uses 460,800. The rest of the system is controlled through the onboard switches on each respective board. The main handles the initial input from MCU_1 before entering the Finite State Machine. Mode 1 handles turning the LED on for a certain color depending on user input (white, purple red, green, blue, yellow, cyan, dark) and changing the brightness of the LED using the built-in hardware Pulse Width Modulation. Mode 2 round-robins the colors in a specific order. It also has the functionality to send the current LED color to the other MCU using SW1 and a hardware interrupt. It also notifies their respective terminal of the current color that is on and who is turn it is to change the LED. To display this, the MCUs use UART0. To make this process easier, flags – specific ASCII bits – are sent from one board to another to signal who should be active and who should be in idle mode. This is also how exiting is managed for all modes, regardless of how the user sends the exit signal. For Mode 3, we simply send flags between the MCU to display who is active/sending a message. To display the sent messages onto their respective terminals, we sent the Ring Buffer between MCUs to the UART0 output. This also ensures we only send 30 chars. In general, this is how the system is designed to ensure requirements are met and constraints are respected. The following call graph displays the functions that are called and what order they're called in.



3. Development Steps and Task Assignments

The first step in the coding portion of our software development is the module test. We tested every module and their functions separately to ensure that we can program it the way we intend to use it later. Thus, we had a module test section for UART 0/5/7 transmit and receive, a section for PWM and LEDs, and a section for the on-board switches. Natasha conducted and created the UART tests, Justin conducted the UART tests, and Hanna did the PWM and LED testing. After testing, we integrated the functions to create the whole system. We did this sequentially and mode-by-mode. After taking care of the modes, we cleaned up the edge cases that we found during testing.

The module test was used to test all of the hardware and software features alongside the custom functions we made. To test the UART 0 module, we sent three ASCII characters. These characters – if working correctly – should display onto the terminal. The terminal's baud rate is set to 460800 for MCU 1 and 921600 for MCU 2. We had an issue with the characters not showing up correctly, but it was an issue with how we handled reading the writing. UART can only read or write from one side. As such, if the terminal is trying to send, it cannot receive at the same time. We changed how it worked testing-wise. One terminal would constantly read/wait and one would be in a "write" or active stage. This made implementation of part 3 very easy. UART 5 was handled by Natasha, and UART 7 was handled by Justin. UART 0 was shared between the two of them. Natasha took care of Ring Buffer as well – alongside the decision to split the ring buffer into two separate structs and pointers. Hanna took care of the PWM, all its functions, and the LED. We tested to make sure that the user's input (a char) can be converted into an int that the PWM function takes to decide what rate it should turn the LED off/on to dim/brighten it. Likewise, the function to change the LED color from a user's input was implemented as well.

The implementation of the modes was done – as previously mentioned – in sequential order. Due to the nature of the module test and the project, Mode 1 and Mode 2 was completed using Hanna's PWM and LED functions. Natasha handled Mode 3 and the general communication between the boards. Justin handled the state machines between the main menu, mode 1, mode 2, and mode 3. The way MCU 2 knew which menu to display was by sending a specific unsigned 8-bit integer signal from MCU 1. Certain ASCII symbols correspond to certain modes for MCU 2 to enter. That's how MCU 2 knew which menu to display when MCU 1 entered different modes. Likewise, exiting a mode sends a signal from MCU 1 to MCU 2 – or in Mode 3's case, from either MCU to the one in a non-transmitting MCU. For Mode 2, the active MCU will send a flag *and* the LED color to the other MCU. The waiting MCU will wait for the flag and then read the next char as the LED color.

Edge cases were mainly handled by Justin and Hanna as they were issues with how the state machine(s) interacted with the actual modes. Most of it was determining if the user's input was valid or not. If the user put in an invalid input for any of the state machines or during the Mode 1 options, the program would ignore it and keep looking for a valid input. An example of this would be putting in a ridiculously large value for Mode 1 Option 2. If a value such as 600 was inputted, the value would overflow and change the brightness. To fix this, Hanna had the system check if the value the user put in was within the range *before* converting from char to unsigned 8-bit integer. Natasha handled clearing the ring buffer – which helped prevent edge cases when switching

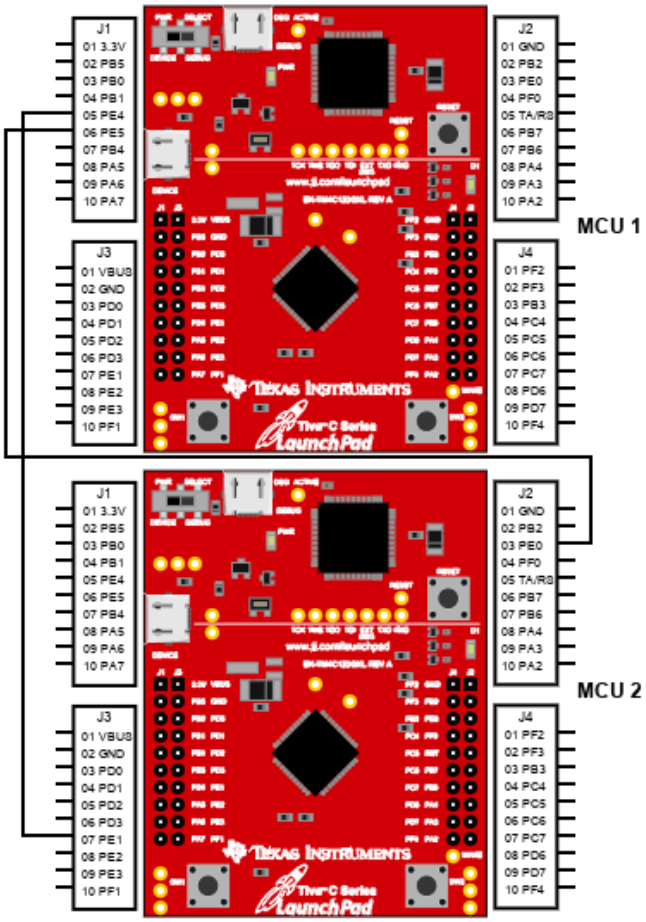
to/from modes and in Mode 3. Everything is cleared from the ring buffers when exiting/entering a mode. For Mode 3, the non-transmitting MCU will have their ring buffer cleared. This prevents an "auto-response" due to the buffer being full when not in transmitting mode.

Task Assignment Table:

Team Member	Role / Responsibility	Assigned Tasks	Completion Date
Natasha Kho	Programmer, Hardware design, debugger, tester	Mainly worked on: Module Test RingBuffer UART Mode 3	03/15/26
Justin Narciso	Programmer, debugger, tester	Mainly worked on: FSMs Mode 2 Hardware	03/15/26
Hanna Estrada	Programmer, debugger, tester	Mainly worked on: Mode 1 PWM + LED Hardware	03/18/26

4. Hardware Design

Schematic

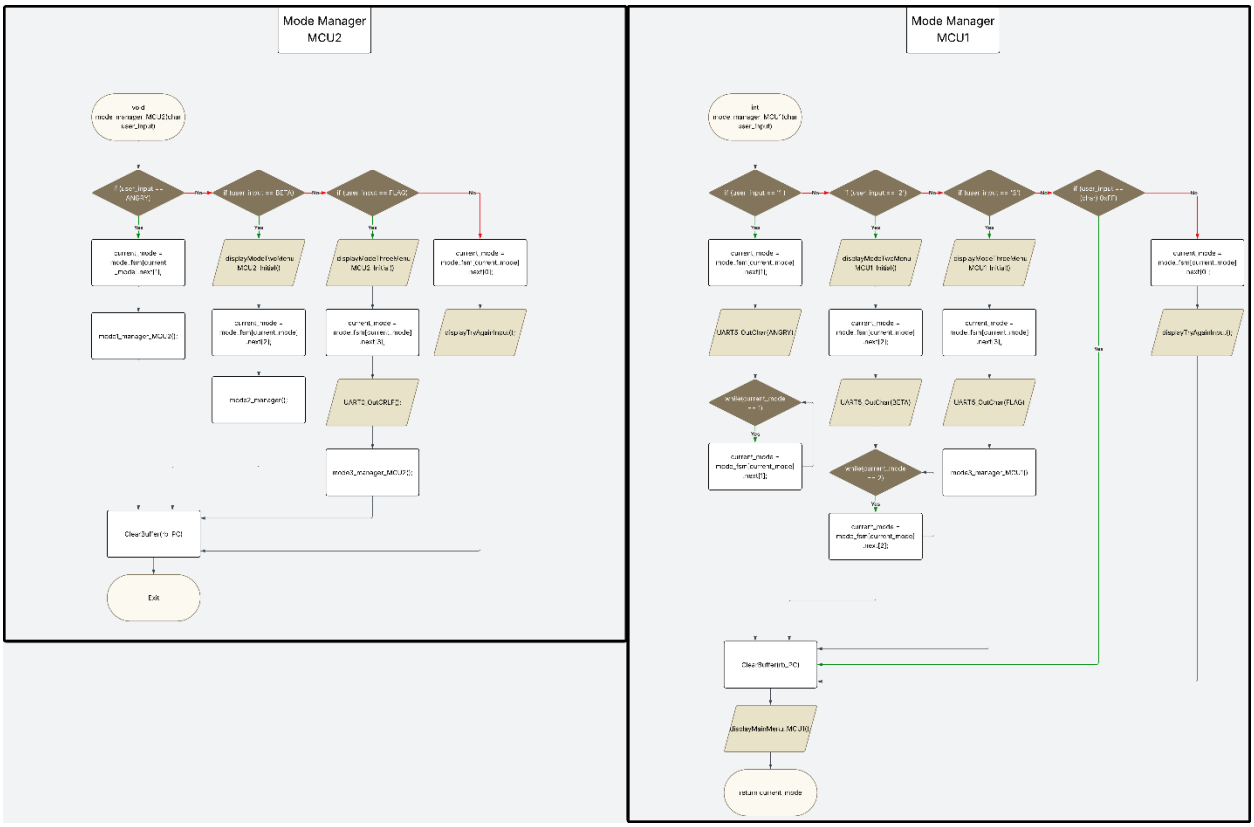


The system for this project is small and inexpensive in terms of amount of materials. To recreate this project, only two boards and a couple jumper wires are required. The wires are connected to the two UART terminals to each other. We used UART 5 and UART 7. We connected the Receiver to the other UART's Transmission. This was PE 4, and PE 5 to PE 0 and PE 1 respectively. No ground or power is required as the boards are connected to the computer – which provides a common ground and power. Likewise, UART 0 does not require any external hardware as it is the same USB to load the code onto the board. The LEDs are PF1-3 and they're onboard. The switches are PF0 andPF4, which are also onboard.

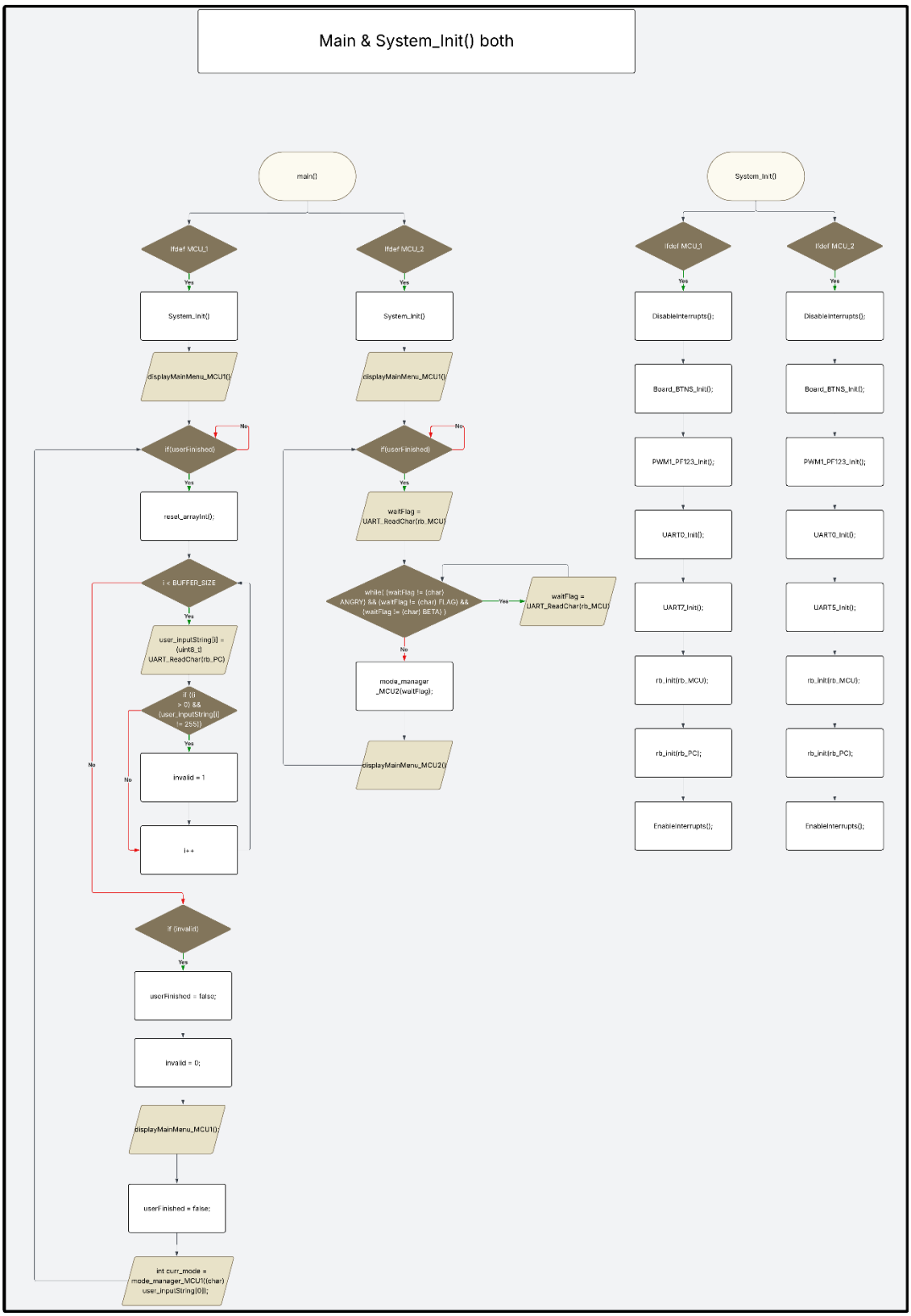
Bill of Materials

Name	Value	Manufacturer	quantity	Description	Comments
EK-TM4C123GXL	22.06	Texas Instruments	2	EVAL KIT TM4C123GXL LAUNCHPAD	
Breadboard Jumper Wires 20cm	6.98	EDGELEC	1	120pcs Breadboard Jumper Wires 20cm Wire Length Optional Dupont Cable Assorted Kit Male to Female Multicolored Ribbon Cables	https://www.amazon.com/dp/B07GD2BWPY?ref=nb_sb_ss_w_as-reorder_k0_1_7&amp=&crid=3FWE1FWMSTDZJ&sprefix=jumper%2B&th=1 huge pack of cables

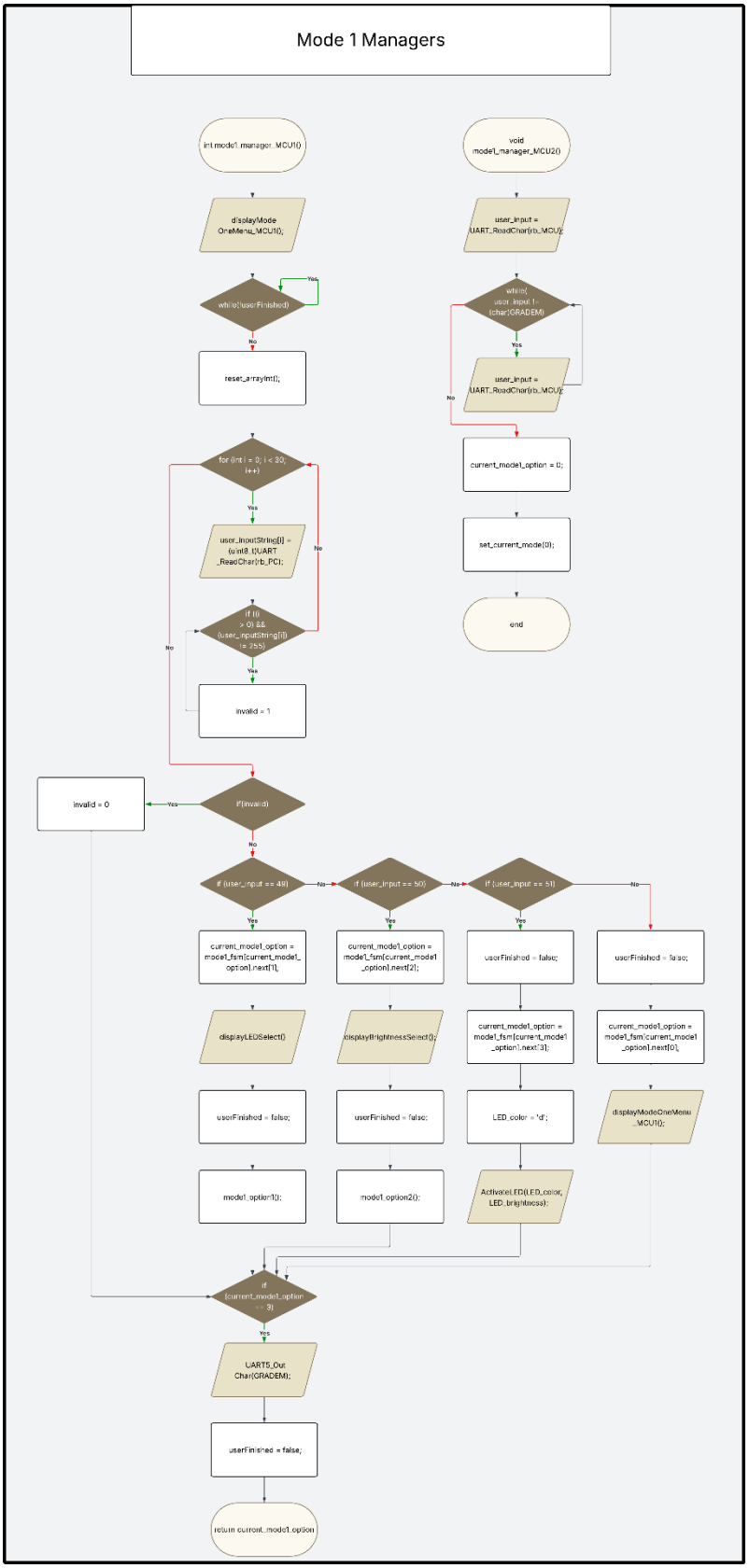
5. Software Design



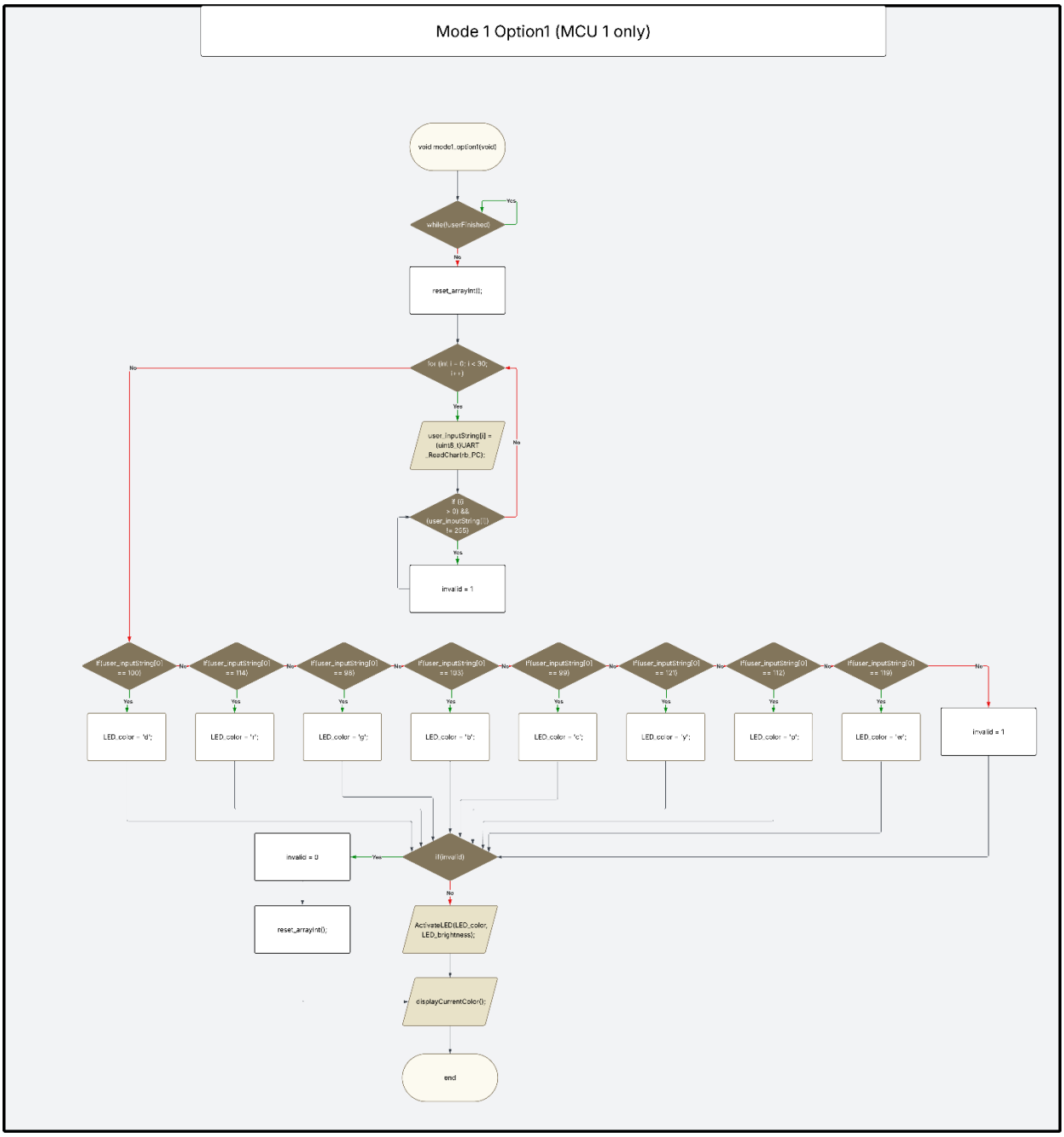
Mode Manager for MCU1 and MCU2



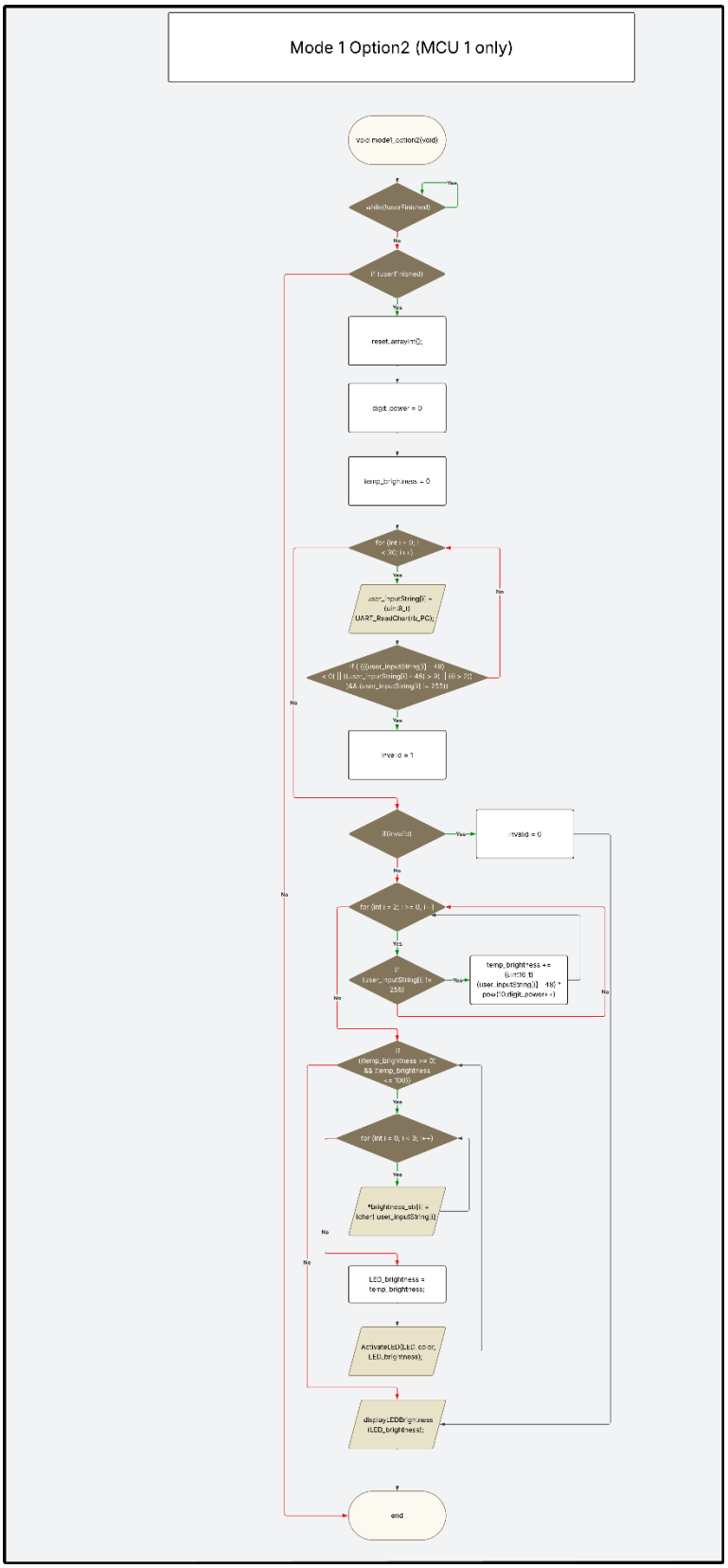
Main and System_init for Both MCUs



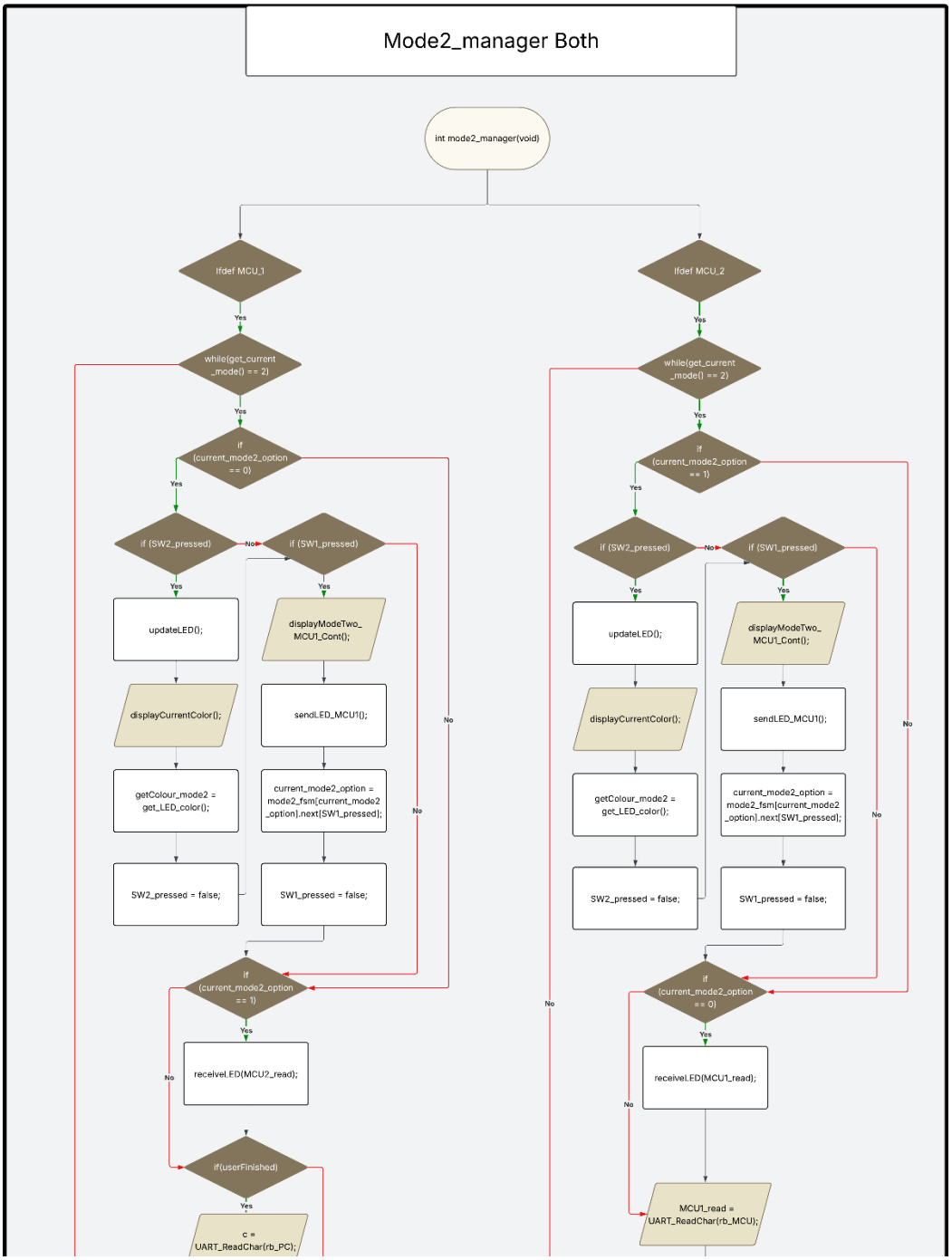
Mode1 Managers for Both

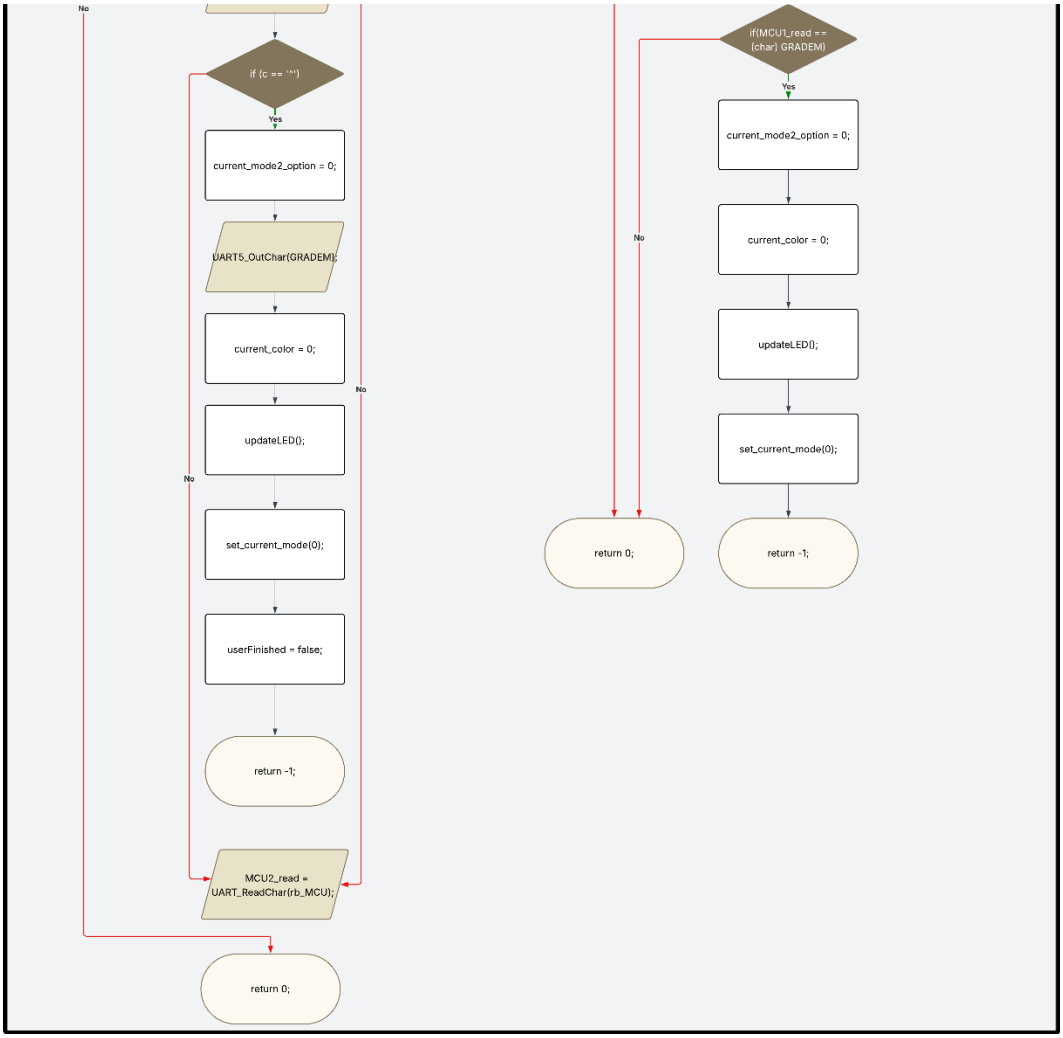


Mode1 option1 MCU1 only

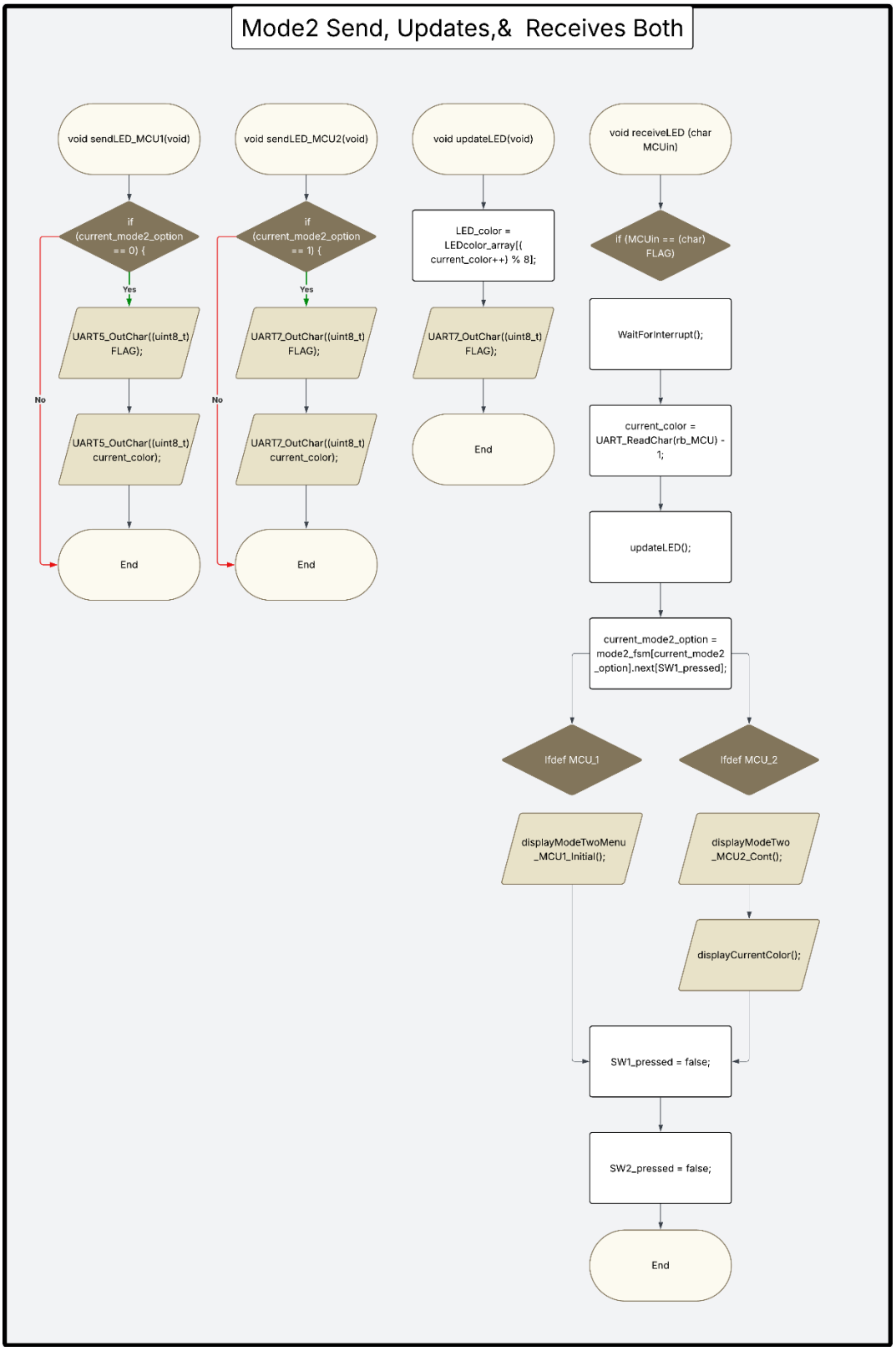


Mode1 option2 MCU1 only

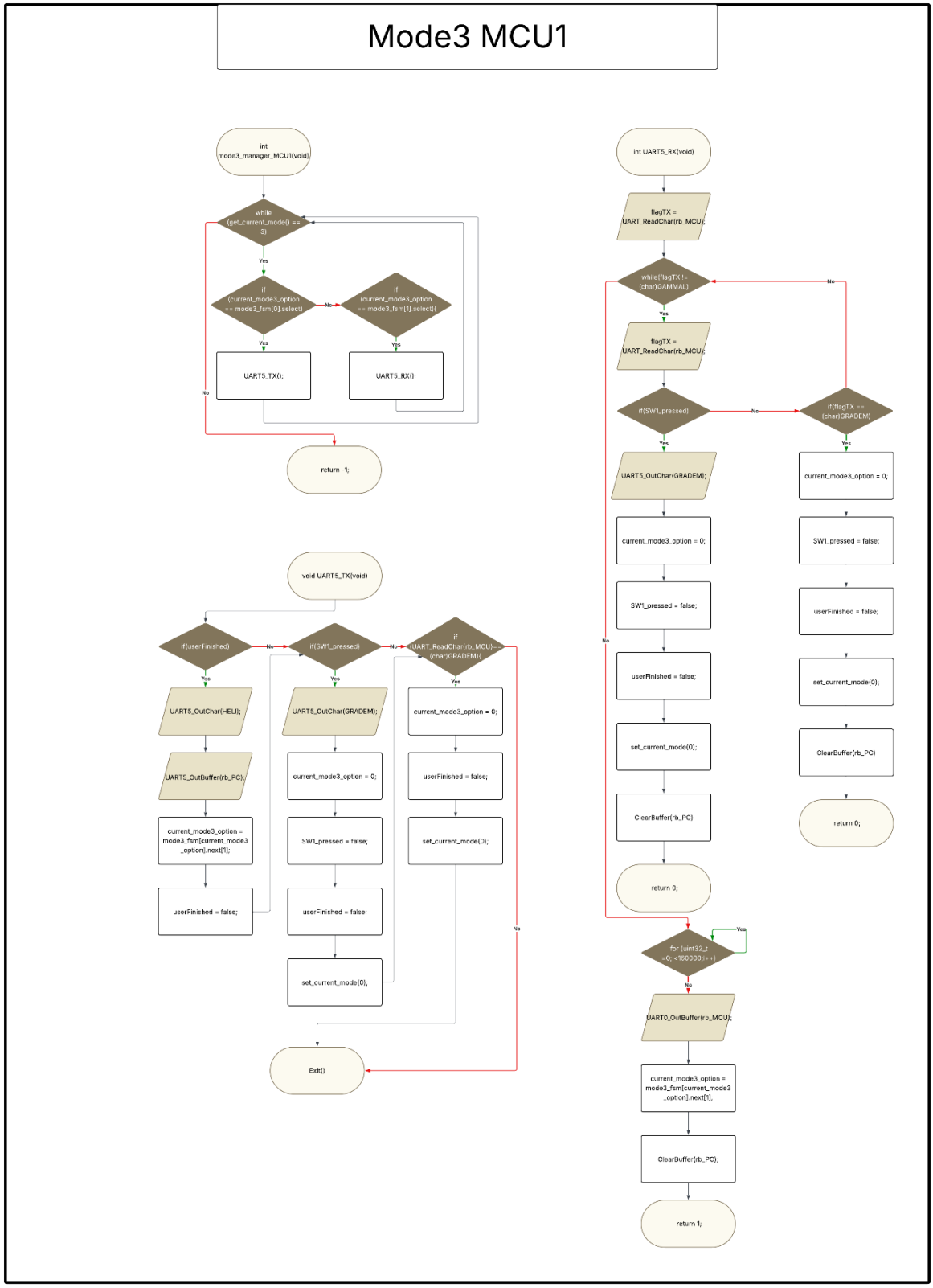

Mode2_manager Both
int mode2_manager(void)
Ifdef MCU_1
Ifdef MCU_2
Yes
while(get_current _mode() == 2)
if (current_mode2_option == 0)
if (current_mode2_option == 1)
if (SW2_pressed)
if (SW1_pressed)
No
updateLED();
displayModeTwo_ MCU1_Cont();
displayCurrentColor();
sendLED_MCU1();
getColour_mode2 = get_LED_color();
current_mode2_option = mode2_fsm[current_mode2 _option].next[SW1_pressed];
SW2_pressed = false;
SW1_pressed = false;
receiveLED(MCU2_read);
if(userFinished)
c = UART_ReadChar(rb_PC);
receiveLED(MCU1_read);
MCU1_read = UART_ReadChar(rb_MCU);



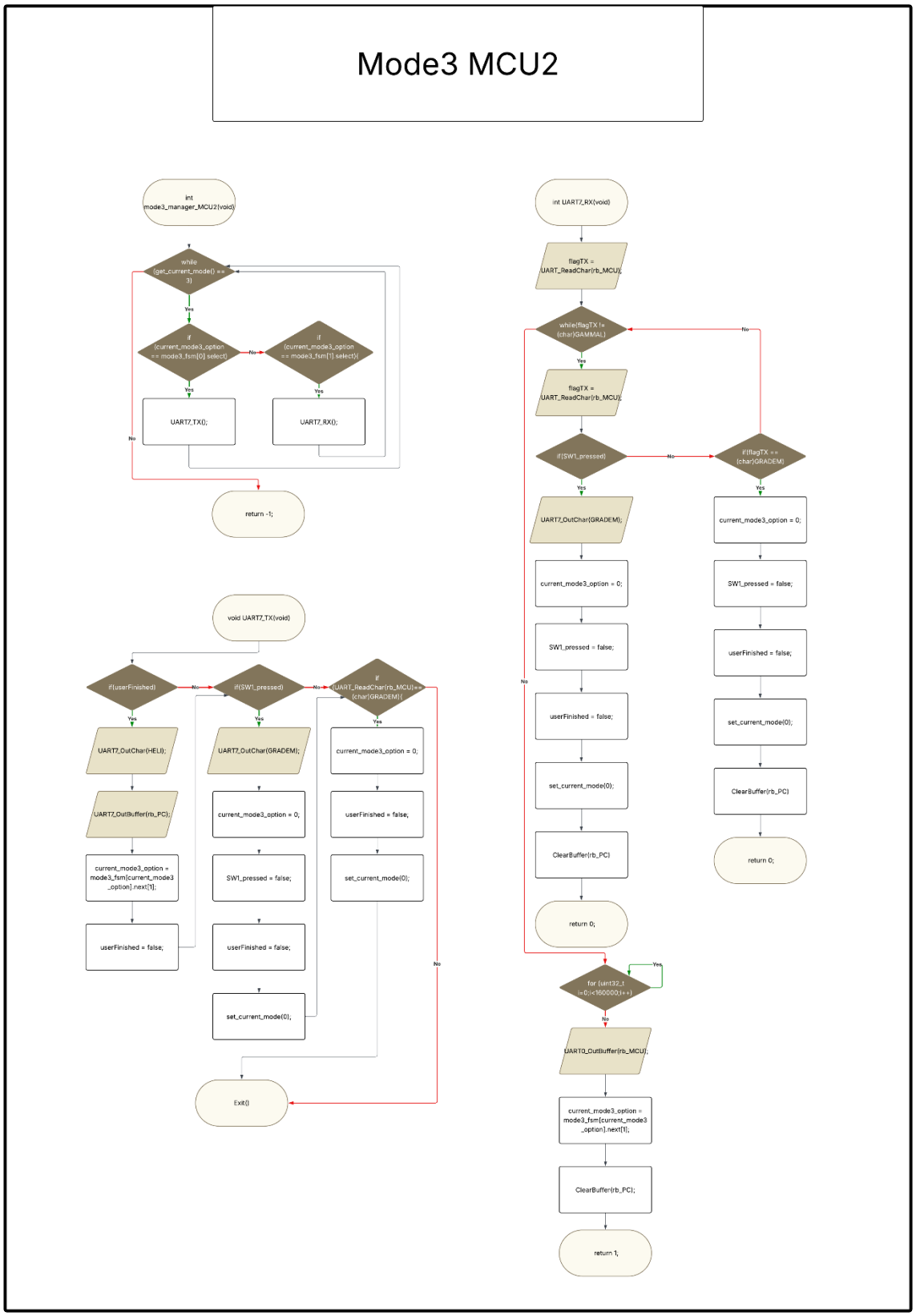
Mode2_manager Both



Mode2 Send, Update, and Receives Both


Mode3 MCU1
int mode3_manager_MCU1(void)
while (get_current_mode() == 3)
Yes
if (current_mode3_option == mode3_fsm[0].select)
No
if (current_mode3_option == mode3_fsm[1].select){
Yes
Yes
UART5_TX();
UART5_RX();
No
return -1;
void UART5_TX(void)
if(userFinished)
No
if(SW1_pressed)
No
if (UART_ReadChar(rb_MCU)== (char)GRADEM){
Yes
Yes
Yes
UART5_OutChar(HELI);
UART5_OutBuffer(rb_PC);
current_mode3_option = mode3_fsm[current_mode3_option].next[1];
userFinished = false;
UART5_OutChar(GRADEM);
current_mode3_option = 0;
SW1_pressed = false;
userFinished = false;
set_current_mode(0);
current_mode3_option = 0;
userFinished = false;
set_current_mode(0);
No
Exit()
int UART5_RX(void)
flagTX = UART_ReadChar(rb_MCU);
while(flagTX != (char)GAMMAL)
No
Yes
flagTX = UART_ReadChar(rb_MCU);
if(SW1_pressed)
No
if(flagTX == (char)GRADEM)
Yes
Yes
UART5_OutChar(GRADEM);
current_mode3_option = 0;
SW1_pressed = false;
userFinished = false;
set_current_mode(0);
ClearBuffer(rb_PC)
return 0;
current_mode3_option = 0;
SW1_pressed = false;
userFinished = false;
set_current_mode(0);
ClearBuffer(rb_PC)
return 0;
No
for (uint32_t i=0;i<160000;i++)
Yes
No
UART0_OutBuffer(rb_MCU);
current_mode3_option = mode3_fsm[current_mode3_option].next[1];
ClearBuffer(rb_PC);
return 1;

Mode3 MCU1


Mode3 MCU2
int mode3_manager_MCU2(void)
while (get_current_mode() == 3)
Yes
if (current_mode3_option == mode3_fsm[0].select)
No
if (current_mode3_option == mode3_fsm[1].select){
Yes
Yes
UART7_TX();
UART7_RX();
No
return -1;
void UART7_TX(void)
if(userFinished)
No
if(SW1_pressed)
No
if (UART_ReadChar(rb_MCU)== (char)GRADEM){
Yes
Yes
Yes
UART7_OutChar(HELI);
UART7_OutBuffer(rb_PC);
current_mode3_option = mode3_fsm[current_mode3_option].next[1];
userFinished = false;
UART7_OutChar(GRADEM);
current_mode3_option = 0;
SW1_pressed = false;
userFinished = false;
set_current_mode(0);
current_mode3_option = 0;
userFinished = false;
set_current_mode(0);
No
Exit()
int UART7_RX(void)
flagTX = UART_ReadChar(rb_MCU);
while(flagTX != (char)GAMMAL)
No
Yes
flagTX = UART_ReadChar(rb_MCU);
if(SW1_pressed)
No
if(flagTX == (char)GRADEM)
Yes
Yes
UART7_OutChar(GRADEM);
current_mode3_option = 0;
SW1_pressed = false;
userFinished = false;
set_current_mode(0);
ClearBuffer(rb_PC)
return 0;
current_mode3_option = 0;
SW1_pressed = false;
No
userFinished = false;
set_current_mode(0);
ClearBuffer(rb_PC)
return 0;
for (uint32_t i=0;i<160000;i++)
Yes
No
UART0_OutBuffer(rb_MCU);
current_mode3_option = mode3_fsm[current_mode3_option].next[1];
ClearBuffer(rb_PC);
return 1;

Mode3 MCU2

In the UART communications project the, there are 5 major files that are responsible for the application level operations. main.c, mode_manager.c, mode1.c, mode2.c, and mode3.c; then also utilize many lower and mid level process like UART0, UART5, UART7, PWM.c, Switches.c, and RingBuffer.c. Some other major general systems to note is the each MCU has 2 ring_buffer systems and two UART systems. One ring buffer is for the MCU to MCU communication, and the other if for the PC to MCU communication. Both MCUs use UART0 for the PC to MCU communication with different baud rates, MCU1 utilizes 921,600 Baudrate, and MCU2 uses to 406,800. MCU1 uses UART5 and MCU2 uses UART7. So when mentioning that it outputs to one sy

Main.c is fairly simple for both MCUs, Both MCUs will display their respective main_menus(MCU1 showing the main_menu, while MCU2 shoing a waiting display. With MCU1 waiting for a valid input which is then entered into mode_manager to determine the mode the system is in. While MCU2 is waiting for a signal from MCU1 to tell exactly what mode MCU2 should operate in.

Mode_manager.c has two forms, both determine what operation their respective MCU is in, but have small differences. MCU1 mode_manager is what takes in a user_input a char, and then determines what mode it enters from there. With invalid inputs simply resulting in mode_manager once more, 1 goes to mode 1, 2 goes to mode 2, and 3 goes to mode 3. Depending on which mode it enter(1,2,3) it will send out a specific signal to notify MCU2 that the mode has changed. This signal is then used to interpret the mode MCU2 and the associated functionalities in each mode. Both then would clear their buffer, and MCU1 would redisplay the main_menu. Both systems utilize a FSM that helps keeps track of the state the program is in and which switch case to enter, these states are the main_menu, mode1, mode2, and mode3.

Mode1.c, there is a major difference between the two MCUs with a majority of the functionality dedicated to MCU1. MCU1 will display the mode1_menu, and check for a valid user_input by reading from the PC buffer. If the read char from the input is invalid it will skip the mode1 operations, and reset invalid to 0, however in the case where the input is valid we begin mode1 operations. If option 1 is selected then it displays the LED_select menu, reset user_finished and enter mode1_option1. If the option2 is selected then displays brightess_select, reset user_finished and enter mode1_option2. If option 3 is selected, user_finished is reset, LED is set to dark. Incase of any other input, mode1 defaults to operations of resetting user_finished, set current_mode1_option back to 0, and redisplay the mode1_menu. mode1_option1() checks if the input is valid from PC1 and changes the LED based on the Char is selected, unless invalid in which LED is remains unaffected. While mode1_option2 checks 3 char input is valid, and is all digits, then takes this and changes the brightness based on the inputted value, and update the LED, and returns to mode1 menu afterward. Afterall these in MCU1, it checks if its mode3, and if it is, all utilized global variables are reset, current_mode is then set back to main_menu, and output a char to MCU2 to signal to exit mode1. While MCU2 simply reads a char constantly and if the char is the exit flag, MCU2 exits mode1. Utilizes an FSM that tracks the

currently selected option, with having 0 as the mode1 menu, option1 for LED color, option2 for LED brightness, and option3 exclusively signaling exit conditions.

Mode2.c for both MCUs are fairly similar to one another, and have the same basic format just with different values and initial set ups and exit conditions. They both will updateLED upon SW2 press this displays the current color selected, and enters the updateLED function which increments through a round-robin of all the colors(dark, red, green, blue, cyan, yellow, pink, white), which then updates the LED. If SW1 pressed the transmitting PC displays that it is now receiving, and then enters the SendLED function. This function transmits the selected LED color to the other MCU via the MCU to MCU UARTs, then enters receiveLED. While the receiveLED with the other MCU which just received with the transmitted color LED, and updates their board, and becomes the transmitter performing in the same manner. The key differences are that MCU1 is starting transmitter, and the MCU1 has exit the system regardless of which MCU is transmitting or not. It performs this exit routine by sending out an exit signal to MCU2 which resets the FSM back to original states, reseting global flags like SW presses, and user_finished is reset, and updates the LED back to dark for both systems. Utilized FSM with two states to determine who is transmitting, with it 0 being MCU1 and 1 is MCU2.

Mode3.c is the System to System two way communication and conversation with both systems be designed in a very similar way just with some key differences. Both MCU systems enter and loop while the current_mode is 3, and then either receives or transmits based on the state the programs in. These if statements then call their respective UART transmit or receive function, with UART5 being for MCU1 and UART7 for MCU2. The transmit funnction has three checks, if the user_finished writing the message, if SW1_presses for exiting, or if an exit flag is received from the other MCU. If user_finished true, transmits a flag to the other MCU to switch to transmission mode, then followed by the string of characters that were entered in the PC. SW1_pressed is true then it sends a exit flag to the other MCU, and resets SW1_pressed, user_finished, mode3 state back to default, and the system mode back to default. Then if exit_flag is read then the it only resets the mode3 state back to default, user_finished is false, and set the current mode back to default. On the other hand, the receive functions wait for a the tranmission character to signal they will transmit soon, if the signal is read then it will then transmit the string received from the other MCU and send it through UART0 to the PC. If it doesn't detect it checks for the exit conditions of SW1_pressed and exit_flag, which are then performs the same operations that are in transmit. Also implemented a 30 character limit on messages so when a message with more than 30 characters is clipped. Utilized FSM with two states to determine who is transmitting, with it 0 being MCU1 and 1 is MCU2.

6. Demonstration

Link to video

For this project, both systems are interconnected in one program and can be demonstrated in the same video.

At the start of the video, both MCUs are reset, which brings up the Main Menu on PC1 and a waiting screen on PC2.

First, the user demonstrates typing in a '1' and clicking enter, which causes PC1 and MCU1 to enter Mode 1. In this mode, PC2 continues to display a waiting screen while PC1 displays a Mode 1 Menu. In this mode, the user begins by typing in a 1 and hitting enter again. This brings up a prompt for the user to enter a color, to which they type 'g' then enter. After hitting enter, the Mode 1 Menu is shown on PC1 again along with the current color, and the LED on the MCU is shown to be green. The user repeats this action, except they enter 'y' and the MCU LED turns yellow. Next, the user enters '2' on the Mode 1 Menu and a prompt to change LED brightness is shown to the user. The user enters the number '50', which causes PC1 to display the current brightness and the Mode 1 Menu again. On the MCU, we can also see the LED dim. The user repeats this action, entering a '10' and we can see the LED dim further as a result. Finally, the user enters '3' on the Mode 1 Menu, which brings the user back to the Main Menu and the Main Menu is shown again on PC1.

Next, the user enters a '2' and enters Mode 2. This mode begins by displaying the color wheel instructions to PC1 and displaying a wait screen to PC2. In this demonstration, the user presses SW2 on MCU1 to change the LED color until it reaches yellow. We can see on the screen that it displays the different LED colors as they change on MCU1. Then, the user presses SW1 on MCU1 and it sends the color yellow over the MCU2 and displays a waiting screen and the current color on PC1. As we can see on PC2, the color wheel instructions are now displayed, and the user then presses SW2 on MCU2 to change colors until the LED is purple, which is shown on PC2's screen. The user then presses SW1 on MCU2 to send purple to MCU1, where we can see the yellow LED change to purple and PC1 updates with the color wheel instructions. The user then demonstrates the whole color wheel on MCU1, looping MCU1's LED colors until it goes from purple to dark twice. The user clicks SW1 on MCU1 to send the color over, then types a '^' into PC1 and clicks enter, which sends both MCU1 and MCU2 back to the main menu, which is displayed again on PC1.

Lastly, the user enters Mode 3 by entering 3 on PC1's main menu and clicking enter. On PC1, the chat room instructions are displayed along with the mode, telling the user to enter a message with less than 30 characters and click enter. It also says that SW1 on the corresponding MCU can be pressed to exit Mode 3 at any time. On PC2, the exit instructions and mode are also displayed, but it says it is waiting for a message from MCU1. Then, on PC1, the user enters 'heello'. This message appears on PC2, and the user can now enter a message on the line below PC1's message. On PC2, the user enters 'hi friend!' and clicks enter. This message then shows up PC1 on a new line, and now PC1 can send another message. The user enters 'good talk :)' on PC1 and hits enter, where it shows up on PC2. Then, the user pressed SW1 on MCU1 to exit Mode 3, which brings back the main menu on PC1 and the waiting screen on PC2. The user then enters Mode 3 again and types a very long

message of ‘aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa’ on PC1 and sends it to PC2, where we can see the message gets cut off after 30 characters on PC2. Then, the user clicks SW1 on MCU2, which also ends Mode 3 and displays the main menu again on PC1. End of demonstration.

8. Conclusion

To conclude this project successfully utilizes the UART0, UART5, UART7, PWM, GPIO buttons, LED to create a UART communication system between two PC and MCU systems. With each UART0 being dedicated to the PC to MCU systems, and the MCU1 using UART5 and MCU2 using UART7 for the MCU to MCU communications.

Successfully implementing and mode manager using a finite state machine to determine what mode is entered based on the user input from the PC terminal. Along with this it properly displays the mode manager menu on the PC displays, and MCU1 correctly communicates to MCU2 when the mode is changed and what mode to enter. This also successfully handles the entering and exiting of the modes.

Then mode1 correctly performed the PC to MCU communication. As mode 1 was successfully implemented and displayed its own menu on PC terminals; and utilized an FSM to determine what submenu to perform based on user input from PC1 terminal. Successfully displays a menu and prompt based on which sub menu was selected. LED color change accepts a char input for color from PC and successfully displaying the color it was changed to on the terminal and LED on the MCU. As well as accepting a 0 to 100 user input so that LED brightness changes successfully dimming or brightening on the board and displaying on the PC terminal. Finally, the Exit condition was successfully handled with MCU1 signaling to MCU2 and resetting the necessary values for the next operation.

Mode 2 successfully implemented the MCU to MCU communications, using its own FSM, and its own entrance by correctly displaying its menu upon entrance on both PCs. With MCU1 initially transmitting and MCU2 initially receiving. Both boards successfully round-robin through the colors upon a SW1 input, displaying the changed color to their respective PC terminal and changing the color on the LED. Then upon SW2 press it correctly changes which MCU is transmitting and receiving and displays so on their respective PC. Then also changes the color of the receiving MCU to what the transmitting MCU had originally displayed. Then successfully exits when user input of '^' is received from PC1, and MCU1 signals MCU2 exit, with both systems resetting variables used for next implementation.

Mode 3 was able to correctly perform the PC to PC communication via the MCUS, using its own FSM to determine which PC is transmitting and receiving based on who transmitted it last. The mode initially begins with PC1 transmitting and PC2 receiving, and the user being prompted to enter a message that would be sent to the other PC. Upon entering the message the other PC1 sends it MCU1 to MCU2 and then displays it on PC2, which will then be able to send their own message. Each message being displayed below the previous message in the terminal. Then if one or the other MCU presses SW1 successfully exiting the mode for both systems. The program also cuts the character limit to 30 and prevents any other character from being sent.

Though successful in our implementation of the whole system in mode manager and all of its modes, there were still issues that we encountered. One was how to handle and read from the ringbuffer. With this design opting for two ringbuffer set up in each MCU to have data be excluded

from each other, and data being directly sent from buffer to transmit. Along with this, there was an issue handling how data was read from the buffer as we struggled with many invalid characters. This was remedies by rewriting the UART handler that would exclude invalid characters and react to certain things in special manner. Another issue was how to format our files for both systems to use them. This was remedied with a config file that would define certain function based on which MCU the user wanted to be in.

We learned how to create and utilize UART and a ringbuffer within our system to communicate system to system to transmit messages and signals. The methods in which taken to trouble shooting a UART systema and it designed. As well as understanding how to handle communication between the two systems to prevent data being lost from conflicting transmissions. Not only that we were able to correctly learn how to use PWM to change brightness of the onboard LED, and create a navigable system based on the inputs of a user on the terminal and how to handle user error within that input.

9. References

He, M. (2025, January). *CECS 447 Lecture 3 - UART Programming* [PowerPoint slides]. CSULB. https://csulb.instructure.com/courses/111080/files/29101229?module_item_id=7003784

He, M. (2026, January). *CECS 447 Lecture - Project 2 Design and Implementation Discussion* [PowerPoint slides]. CSULB. https://csulb.instructure.com/courses/111080/files/29824112?module_item_id=7055083

xHe, M. (2026, January). *CECS 447 Lecture - Module Design and Module Test* [PowerPoint slides]. CSULB. https://csulb.instructure.com/courses/111080/files/29794016?module_item_id=7053243

xHe, M. (2025, December). *CECS 347 Lecture 4 – UART* [PowerPoint slides]. CSULB. https://csulb.instructure.com/courses/111080/files/29102121?module_item_id=7003850

xHe, M. (2025 December). *Lecture 2 - PWM* [PowerPoint slides]. CSULB. https://csulb.instructure.com/courses/111080/files/27997739?module_item_id=6630011

Phillips Semiconductors. (1999, April 12th). DATA SHEET PCD8554 48 × 84 pixels matrix LCD controller/driver. Phillips Semiconductors. https://cdn.sparkfun.com/assets/b/1/b/e/f/Nokia5110.pdf

Texas Instruments. (2014, June 12th). Tiva™ TM4C123GH6PM Microcontroller DATA SHEET. Texas Instruments. https://www.ti.com/lit/ds/symlink/tm4c123gh6pm.pdf

Valvano J. & Yerraballi R. (2022). Chapter 8: Analog to Digital Conversion, Data Acquisition, and Control. University of Texas. https://users.ece.utexas.edu/~valvano/Volume1/IntroToEmbSys/Ch8_ADC.htm